

DCC - GARCH

1. Load Packages

```
packages <- c("readr", "dplyr", "lubridate", "rugarch", "rmgarch",
             "xts", "zoo", "ggplot2", "tidyr", "gt")

# Load the packages listed above.
lapply(packages, library, character.only = TRUE)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

  filter, lag

The following objects are masked from 'package:base':

  intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

  date, intersect, setdiff, union

Loading required package: parallel

Attaching package: 'rmgarch'

The following objects are masked from 'package:dplyr':

  first, last

Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

  as.Date, as.Date.numeric

##### Warning from 'xts' package #####
#####
#
```

```

#
# The dplyr lag() function breaks how base R's lag() function is supposed to
# work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or
# source() into this session won't work correctly.
#
#
# Use stats::lag() to make sure you're not using dplyr::lag(), or you can add
# conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop
# dplyr from breaking base R's lag() function.
#
#
# Code in packages is not affected. It's protected by R's namespace mechanism
# Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning.
#
#####
#####

```

Attaching package: 'xts'

The following objects are masked from 'package:rmgarch':

first, last

The following objects are masked from 'package:dplyr':

first, last

```

[[1]]
[1] "readr"      "stats"      "graphics"   "grDevices" "utils"      "datasets"
[7] "methods"    "base"

```

```

[[2]]
[1] "dplyr"      "readr"      "stats"      "graphics"   "grDevices" "utils"
[7] "datasets"   "methods"    "base"

```

```

[[3]]
[1] "lubridate" "dplyr"      "readr"      "stats"      "graphics"   "grDevices"

```

```

[7] "utils"      "datasets"  "methods"   "base"

[[4]]
[1] "rugarch"    "parallel"  "lubridate" "dplyr"     "readr"     "stats"
"
[7] "graphics"   "grDevices" "utils"     "datasets"  "methods"   "base"

[[5]]
[1] "rmgarch"    "rugarch"   "parallel"  "lubridate" "dplyr"     "readr"
"
[7] "stats"      "graphics"  "grDevices" "utils"     "datasets"  "metho
ds"
[13] "base"

[[6]]
[1] "xts"        "zoo"       "rmgarch"   "rugarch"   "parallel"  "lubri
date"
[7] "dplyr"      "readr"     "stats"     "graphics"  "grDevices" "utils"
"
[13] "datasets"   "methods"   "base"

[[7]]
[1] "xts"        "zoo"       "rmgarch"   "rugarch"   "parallel"  "lubri
date"
[7] "dplyr"      "readr"     "stats"     "graphics"  "grDevices" "utils"
"
[13] "datasets"   "methods"   "base"

[[8]]
[1] "ggplot2"    "xts"       "zoo"       "rmgarch"   "rugarch"   "paral
lel"
[7] "lubridate"  "dplyr"     "readr"     "stats"     "graphics"  "grDev
ices"
[13] "utils"      "datasets"  "methods"   "base"

[[9]]
[1] "tidyr"      "ggplot2"   "xts"       "zoo"       "rmgarch"   "rugar
ch"
[7] "parallel"   "lubridate" "dplyr"     "readr"     "stats"     "graph
ics"
[13] "grDevices"  "utils"     "datasets"  "methods"   "base"

[[10]]
[1] "gt"         "tidyr"     "ggplot2"   "xts"       "zoo"       "rmgar
ch"
[7] "rugarch"    "parallel"  "lubridate" "dplyr"     "readr"     "stats"
"

```

```
[13] "graphics" "grDevices" "utils" "datasets" "methods" "base"
```

2. Import Daily Index Price Data

The extended model uses the same common six-market trading-date sample as the baseline model. This keeps the sample period and observation count identical across the two DCC-GARCH specifications.

The repository includes a frozen local dataset ending on 2023-08-31. Rendering this document does not download or update market data.

```
data_candidates <- c(
  "../dataset/outputs/daily_prices_all_markets.csv",
  "dataset/outputs/daily_prices_all_markets.csv",
  "daily_prices_all_markets.csv"
)
data_path <- data_candidates[file.exists(data_candidates)][1]

if (is.na(data_path)) {
  stop("The frozen daily_prices_all_markets.csv included with the project was not found.")
}
```

```
daily_prices <- readr::read_csv(data_path)
```

Rows: 4296 Columns: 7

— Column specification —

Delimiter: ","

dbl (6): STI, Hang_Seng, KOSPI, Nikkei_225, Shanghai_Composite, SP500
date (1): Date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
head(daily_prices)
```

A tibble: 6 × 7

	Date	STI	Hang_Seng	KOSPI	Nikkei_225	Shanghai_Composite	SP500
	<date>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2003-01-06	1332.	9666.	667.	8713.	1335.	929.
2	2003-01-07	1319.	9652.	652.	8656.	1332.	923.
3	2003-01-08	1332.	9688.	652.	8518.	1372.	910.
4	2003-01-09	1335.	9675.	630.	8498.	1397.	928.
5	2003-01-10	1347.	9722.	628.	8470.	1385.	928.
6	2003-01-14	1401.	9796.	650.	8553.	1467.	932.

```
str(daily_prices)
```

```

spc_tbl_ [4,296 × 7] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ Date           : Date[1:4296], format: "2003-01-06" "2003-01-07"
 ...
 $ STI            : num [1:4296] 1332 1319 1332 1335 1347 ...
 $ Hang_Seng      : num [1:4296] 9666 9652 9688 9675 9722 ...
 $ KOSPI          : num [1:4296] 667 652 652 630 628 ...
 $ Nikkei_225     : num [1:4296] 8713 8656 8518 8498 8470 ...
 $ Shanghai_Composite: num [1:4296] 1335 1332 1372 1397 1385 ...
 $ SP500          : num [1:4296] 929 923 910 928 928 ...
 - attr(*, "spec")=
 .. cols(
 ..   Date = col_date(format = ""),
 ..   STI = col_double(),
 ..   Hang_Seng = col_double(),
 ..   KOSPI = col_double(),
 ..   Nikkei_225 = col_double(),
 ..   Shanghai_Composite = col_double(),
 ..   SP500 = col_double()
 .. )
 - attr(*, "problems")=<externalptr>

```

3. Clean Date and Restrict Sample Period

```

daily_prices <- daily_prices %>%
  mutate(Date = as.Date(Date)) %>%
  arrange(Date) %>%
  filter(Date >= as.Date("2003-01-01"),
         Date <= as.Date("2023-08-31"))

```

```
head(daily_prices)
```

```

# A tibble: 6 × 7
  Date           STI Hang_Seng KOSPI Nikkei_225 Shanghai_Composite SP500
  <date>         <dbl>   <dbl> <dbl>   <dbl>         <dbl> <dbl>
1 2003-01-06 1332.     9666.  667.    8713.         1335.  929.
2 2003-01-07 1319.     9652.  652.    8656.         1332.  923.
3 2003-01-08 1332.     9688.  652.    8518.         1372.  910.
4 2003-01-09 1335.     9675.  630.    8498.         1397.  928.
5 2003-01-10 1347.     9722.  628.    8470.         1385.  928.
6 2003-01-14 1401.     9796.  650.    8553.         1467.  932.

```

```
tail(daily_prices)
```

```

# A tibble: 6 × 7
  Date           STI Hang_Seng KOSPI Nikkei_225 Shanghai_Composite SP500
  <date>         <dbl>   <dbl> <dbl>   <dbl>         <dbl> <dbl>
1 2023-08-24 3181.    18212. 2538.   32287.         3082. 4376.
2 2023-08-25 3190.    18119. 2519.   31624.         3064. 4406.
3 2023-08-28 3214.    18131. 2543.   32170.         3099. 4433.
4 2023-08-29 3223.    18484. 2552.   32227.         3136. 4498.

```

```
5 2023-08-30 3220.    18483. 2561.    32333.    3137. 4515.
6 2023-08-31 3233.    18382. 2556.    32619.    3120. 4508.
```

```
summary(daily_prices)
```

Date	STI	Hang_Seng	KOSPI
Min. :2003-01-06	Min. :1214	Min. : 8409	Min. : 515.2
1st Qu.:2008-03-02	1st Qu.:2593	1st Qu.:18734	1st Qu.:1481.3
Median :2013-04-24	Median :3043	Median :21905	Median :1958.2
Mean :2013-05-02	Mean :2852	Mean :21301	Mean :1863.5
3rd Qu.:2018-07-05	3rd Qu.:3230	3rd Qu.:24616	3rd Qu.:2161.0
Max. :2023-08-31	Max. :3876	Max. :33154	Max. :3305.2

Nikkei_225	Shanghai_Composite	SP500
Min. : 7055	Min. :1012	Min. : 676.5
1st Qu.:10773	1st Qu.:2094	1st Qu.:1224.3
Median :15883	Median :2862	Median :1586.4
Mean :16669	Mean :2700	Mean :2046.7
3rd Qu.:21318	3rd Qu.:3247	3rd Qu.:2713.0
Max. :33753	Max. :6092	Max. :4793.5

```
colSums(is.na(daily_prices))
```

	Date	STI	Hang_Seng	KOSPI
OSPI				
	0	0	0	
	Nikkei_225	Shanghai_Composite	SP500	
	0	0	0	

```
sum(duplicated(daily_prices$Date))
```

```
[1] 0
```

```
nrow(daily_prices)
```

```
[1] 4296
```

4. Calculate Daily Log Returns

```
daily_returns <- daily_prices %>%
  mutate(
    r_STI = (log(STI) - log(lag(STI))) * 100,
    r_Hang_Seng = (log(Hang_Seng) - log(lag(Hang_Seng))) * 100,
    r_KOSPI = (log(KOSPI) - log(lag(KOSPI))) * 100,
    r_Nikkei_225 = (log(Nikkei_225) - log(lag(Nikkei_225))) * 100,
    r_SP500 = (log(SP500) - log(lag(SP500))) * 100,
    r_Shanghai = (log(Shanghai_Composite) - log(lag(Shanghai_Composit
e))) * 100
  ) %>%
  select(
    Date,
    r_Shanghai,
```

```

    r_Hang_Seng,
    r_Nikkei_225,
    r_KOSPI,
    r_STI,
    r_SP500
  ) %>%
  na.omit()

nrow(daily_returns)

[1] 4295

colSums(is.na(daily_returns))

      Date  r_Shanghai r_Hang_Seng r_Nikkei_225      r_KOSPI
r_STI
      0              0              0              0              0
      0
r_SP500
      0

```

4.1 Daily Data Audit and Descriptive Statistics

The audit table records how the common daily price sample becomes the return sample used in the extended DCC-GARCH estimation. The descriptive table includes the five Asian return series and the S&P 500 return.

```

daily_data_audit <- data.frame(
  Stage = c("Common six-market daily prices", "Extended daily returns"),
  Observations = c(nrow(daily_prices), nrow(daily_returns)),
  Start_Date = c(as.character(min(daily_prices$Date)), as.character(min(
    daily_returns$Date))),
  End_Date = c(as.character(max(daily_prices$Date)), as.character(max(d
    aily_returns$Date))),
  Missing_Values = c(sum(is.na(daily_prices)), sum(is.na(daily_return
    s))),
  Duplicate_Dates = c(sum(duplicated(daily_prices$Date)), sum(duplicate
    d(daily_returns$Date)))
)

```

```

daily_data_audit

      Stage Observations Start_Date  End_Date
1 Common six-market daily prices      4296 2003-01-06 2023-08-31
2      Extended daily returns      4295 2003-01-07 2023-08-31
  Missing_Values Duplicate_Dates
1              0              0
2              0              0

```

```

return_variables <- setdiff(names(daily_returns), "Date")

daily_descriptive_statistics <- do.call(rbind, lapply(return_variables,
function(variable) {
  x <- daily_returns[[variable]]
  n <- length(x)
  x_mean <- mean(x)
  x_sd <- sd(x)
  skewness <- mean((x - x_mean)^3) / x_sd^3
  excess_kurtosis <- mean((x - x_mean)^4) / x_sd^4 - 3
  jb_statistic <- n / 6 * (skewness^2 + excess_kurtosis^2 / 4)

  data.frame(
    Variable = variable,
    N = n,
    Mean = x_mean,
    Std_Dev = x_sd,
    Minimum = min(x),
    Maximum = max(x),
    Skewness = skewness,
    Excess_Kurtosis = excess_kurtosis,
    JB_Statistic = jb_statistic,
    JB_p_value = pchisq(jb_statistic, df = 2, lower.tail = FALSE)
  )
}))

```

```
daily_descriptive_statistics
```

	Variable	N	Mean	Std_Dev	Minimum	Maximum	Skewness
1	r_Shanghai	4295	0.01976957	1.647485	-12.76357	9.034251	-0.41447743
2	r_Hang_Seng	4295	0.01496542	1.576037	-14.69537	14.243074	0.01113167
3	r_Nikkei_225	4295	0.03073461	1.561364	-12.92448	13.666939	-0.47585199
4	r_KOSPI	4295	0.03129102	1.377895	-11.17200	11.284352	-0.53030364
5	r_STI	4295	0.02065441	1.165765	-10.62797	15.471889	0.35445471
6	r_SP500	4295	0.03677332	1.301265	-13.79886	10.423558	-0.76357519
	Excess_Kurtosis	JB_Statistic	JB_p_value				
1	5.380329	5303.449	0				
2	10.110441	18293.392	0				
3	9.215200	15359.217	0				
4	7.491423	10244.703	0				
5	15.618838	43746.483	0				
6	13.420527	32649.650	0				

5. Export Cleaned Daily Returns

```
write.csv(daily_returns, "daily_returns_for_dcc.csv", row.names = FALSE)
```


6. Estimate DCC-GARCH Model

```
# Keep only return variables for DCC-GARCH
dcc_data_with_sp500 <- daily_returns %>%
  select(
    r_Shanghai,
    r_Hang_Seng,
    r_Nikkei_225,
    r_KOSPI,
    r_STI,
    r_SP500
  )

# Check data
head(dcc_data_with_sp500)

# A tibble: 6 × 6
  r_Shanghai r_Hang_Seng r_Nikkei_225 r_KOSPI r_STI r_SP500
    <dbl>      <dbl>      <dbl>    <dbl> <dbl>    <dbl>
1   -0.196    -0.140    -0.654 -2.20  -0.972   -0.657
2    2.96     0.370    -1.62  -0.0736  1.02    -1.42
3    1.83    -0.132    -0.234 -3.33   0.208    1.92
4   -0.900     0.475    -0.324 -0.324  0.901     0
5    5.75     0.767     0.971  3.39   3.94    0.440
6   -0.474     0.785     0.684 -0.271  -1.06   -1.45

summary(dcc_data_with_sp500)
```

r_Shanghai		r_Hang_Seng		r_Nikkei_225	
Min.	:-12.76357	Min.	:-14.69537	Min.	:-12.92448
1st Qu.	:-0.70900	1st Qu.	:-0.69851	1st Qu.	:-0.69144
Median	: 0.03665	Median	: 0.05591	Median	: 0.06842
Mean	: 0.01977	Mean	: 0.01496	Mean	: 0.03073
3rd Qu.	: 0.78061	3rd Qu.	: 0.76043	3rd Qu.	: 0.83989
Max.	: 9.03425	Max.	: 14.24307	Max.	: 13.66694

r_KOSPI		r_STI		r_SP500	
Min.	:-11.17200	Min.	:-10.62797	Min.	:-13.79886
1st Qu.	:-0.56747	1st Qu.	:-0.48536	1st Qu.	:-0.44492
Median	: 0.07836	Median	: 0.04508	Median	: 0.08312
Mean	: 0.03129	Mean	: 0.02065	Mean	: 0.03677
3rd Qu.	: 0.71746	3rd Qu.	: 0.52959	3rd Qu.	: 0.60207
Max.	: 11.28435	Max.	: 15.47189	Max.	: 10.42356

7. Set up DCC-GARCH Specification

```
# Univariate GARCH(1,1) specification for each market
uspec_with_sp500 <- ugarchspec(
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE),
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  distribution.model = "norm"
)
```

```
# Apply the same GARCH specification to all return series
multispec_model_with_sp500 <- multispec(replicate(ncol(dcc_data_with_sp
500), uspec_with_sp500))

# DCC-GARCH(1,1) specification
dcc_spec_with_sp500 <- dccspec(
  uspec = multispec_model_with_sp500,
  dccOrder = c(1, 1),
  distribution = "mvnrm"
)
```

8. Fit DCC-GARCH Model

```
dcc_fit_with_sp500 <- dccfit(
  dcc_spec_with_sp500,
  data = dcc_data_with_sp500
)

# Create a clean parameter table
dcc_param_table_with_sp500 <- as.data.frame(dcc_fit_with_sp500@mfit$mat
coef)

# Add parameter names
dcc_param_table_with_sp500$Parameter <- rownames(dcc_param_table_with_s
p500)

# Rename columns manually
colnames(dcc_param_table_with_sp500) <- c(
  "Estimate",
  "Std_Error",
  "t_value",
  "p_value",
  "Parameter"
)

# Reorder columns and add confidence intervals
dcc_param_table_with_sp500 <- dcc_param_table_with_sp500 %>%
  select(Parameter, Estimate, Std_Error, t_value, p_value) %>%
  mutate(
    CI_Lower = Estimate - 1.96 * Std_Error,
    CI_Upper = Estimate + 1.96 * Std_Error
  )

# Display as a clean table
dcc_param_gt_with_sp500 <- dcc_param_table_with_sp500 %>%
  gt::gt() %>%
  gt::tab_header(
    title = "DCC-GARCH Parameter Estimates"
```

```

) %>%
gt::cols_label(
  Parameter = "Parameter",
  Estimate = "Estimate",
  Std_Error = "Std. Error",
  t_value = "t value",
  p_value = "p-value",
  CI_Lower = "95% CI Lower",
  CI_Upper = "95% CI Upper"
) %>%
gt::fmt_number(
  columns = c(Estimate, Std_Error, t_value, p_value, CI_Lower, CI_Upper),
  decimals = 6
)
dcc_param_gt_with_sp500

```

Table 1: DCC-GARCH Parameter Estimates

Parameter	Estimate	Std. Error	t value	p-value	95% CI Lower	95% CI Upper
[r_Shanghai].mu	0.023693	0.019351	1.224376	0.220810	-0.014235	0.061621
[r_Shanghai].omega	0.015342	0.007275	2.108825	0.034960	0.001083	0.029602
[r_Shanghai].alpha1	0.063647	0.014207	4.479839	0.000007	0.035801	0.091494
[r_Shanghai].beta1	0.932926	0.014546	64.137809	0.000000	0.904416	0.961435
[r_Hang_Seng].mu	0.054207	0.019007	2.852029	0.004344	0.016954	0.091460
[r_Hang_Seng].omega	0.031903	0.009856	3.236763	0.001209	0.012584	0.051221
[r_Hang_Seng].alpha1	0.072191	0.011207	6.441668	0.000000	0.050226	0.094157
[r_Hang_Seng].beta1	0.913969	0.012969	70.473096	0.000000	0.888549	0.939388
[r_Nikkei_225].mu	0.077844	0.019203	4.053687	0.000050	0.040205	0.115482
[r_Nikkei_225].omega	0.068653	0.018888	3.634683	0.000278	0.031632	0.105675
[r_Nikkei_225].alpha1	0.112878	0.017982	6.277261	0.000000	0.077633	0.148123
[r_Nikkei_225].beta1	0.860088	0.021517	39.973073	0.000000	0.817915	0.902260
[r_KOSPI].mu	0.045109	0.016013	2.817039	0.004847	0.013724	0.076494
[r_KOSPI].omega	0.025772	0.008251	3.123422	0.001788	0.009600	0.041944
[r_KOSPI].alpha1	0.076797	0.012473	6.157051	0.000000	0.052350	0.101244
[r_KOSPI].beta1	0.908106	0.014874	61.052201	0.000000	0.878952	0.937259
[r_STI].mu	0.042122	0.012680	3.321961	0.000894	0.017269	0.066974
[r_STI].omega	0.017094	0.005301	3.224671	0.001261	0.006704	0.027485
[r_STI].alpha1	0.085611	0.015699	5.453391	0.000000	0.054842	0.116381
[r_STI].beta1	0.901263	0.016764	53.762345	0.000000	0.868406	0.934121
[r_SP500].mu	0.075648	0.013383	5.652719	0.000000	0.049418	0.101878
[r_SP500].omega	0.039484	0.009135	4.322479	0.000015	0.021580	0.057387
[r_SP500].alpha1	0.132526	0.017569	7.543102	0.000000	0.098091	0.166962
[r_SP500].beta1	0.840679	0.018282	45.985095	0.000000	0.804847	0.876510
[Joint]dcc1	0.011281	0.001840	6.130687	0.000000	0.007674	0.014887
[Joint]dcc1	0.978109	0.004618	211.816078	0.000000	0.969059	0.987160

```
gt::gtsave(
  dcc_param_gt_with_sp500,
  filename = "dcc_garch_parameter_estimates.png"
)
```

file:///C:/Users/20945/AppData/Local/Temp/RtmpUJvQ2V/file5aac6264299d.html screenshot completed

9. Extract Daily Dynamic Correlations

```
# Extract dynamic correlation array
dcc_cor_with_sp500 <- rcor(dcc_fit_with_sp500)

avg_dcc_cor_matrix_with_sp500 <- apply(dcc_cor_with_sp500, c(1, 2), mean, na.rm = TRUE)

colnames(avg_dcc_cor_matrix_with_sp500) <- colnames(dcc_data_with_sp500)
```

```
rownames(avg_dcc_cor_matrix_with_sp500) <- colnames(dcc_data_with_sp500)

knitr::kable(
  avg_dcc_cor_matrix_with_sp500,
  digits = 4,
  caption = "Average Dynamic Conditional Correlation Matrix"
)
```

Average Dynamic Conditional Correlation Matrix

	r_Shanghai	r_Hang_Seng	r_Nikkei_225	r_KOSPI	r_STI	r_SP500
	i	g		PI		0
r_Shanghai	1.0000	0.4973	0.2776	0.3063	0.3070	0.1231
r_Hang_Seng	0.4973	1.0000	0.5429	0.6139	0.6623	0.2610
r_Nikkei_225	0.2776	0.5429	1.0000	0.6232	0.5547	0.2310
r_KOSPI	0.3063	0.6139	0.6232	1.0000	0.5691	0.2502
r_STI	0.3070	0.6623	0.5547	0.5691	1.0000	0.2818
r_SP500	0.1231	0.2610	0.2310	0.2502	0.2818	1.0000

```
# Check dimensions
dim(dcc_cor_with_sp500)

[1] 6 6 4295

## Stata-style Pairwise Dynamic Correlation Table
make_corr_row <- function(x, pair_name) {
  x <- na.omit(x)
  coef <- mean(x)
  se <- sd(x) / sqrt(length(x))
  z_value <- coef / se
  p_value <- 2 * (1 - pnorm(abs(z_value)))
  ci_lower <- coef - 1.96 * se
  ci_upper <- coef + 1.96 * se

  data.frame(
    Pair = pair_name,
    Coef = coef,
    Std_Error = se,
    z = z_value,
    P_value = p_value,
    CI_Lower = ci_lower,
    CI_Upper = ci_upper
  )
}
```

```

    )
  }

pairwise_corr_table <- rbind(
  make_corr_row(dcc_cor_with_sp500[1, 2, ], "corr(r_SH, r_Hang_Seng)"),
  make_corr_row(dcc_cor_with_sp500[1, 3, ], "corr(r_SH, r_Nikkei)"),
  make_corr_row(dcc_cor_with_sp500[1, 4, ], "corr(r_SH, r_KOSPI)"),
  make_corr_row(dcc_cor_with_sp500[1, 5, ], "corr(r_SH, r_STI)"),
  make_corr_row(dcc_cor_with_sp500[1, 6, ], "corr(r_SH, r_SP500)"),
  make_corr_row(dcc_cor_with_sp500[2, 3, ], "corr(r_Hang_Seng, r_Nikkei)"),
  ),
  make_corr_row(dcc_cor_with_sp500[2, 4, ], "corr(r_Hang_Seng, r_KOSPI)"),
  ),
  make_corr_row(dcc_cor_with_sp500[2, 5, ], "corr(r_Hang_Seng, r_STI)"),
  make_corr_row(dcc_cor_with_sp500[2, 6, ], "corr(r_Hang_Seng, r_SP500)"),
  ),
  make_corr_row(dcc_cor_with_sp500[3, 4, ], "corr(r_Nikkei, r_KOSPI)"),
  make_corr_row(dcc_cor_with_sp500[3, 5, ], "corr(r_Nikkei, r_STI)"),
  make_corr_row(dcc_cor_with_sp500[3, 6, ], "corr(r_Nikkei, r_SP500)"),
  make_corr_row(dcc_cor_with_sp500[4, 5, ], "corr(r_KOSPI, r_STI)"),
  make_corr_row(dcc_cor_with_sp500[4, 6, ], "corr(r_KOSPI, r_SP500)"),
  make_corr_row(dcc_cor_with_sp500[5, 6, ], "corr(r_STI, r_SP500)")
)

pairwise_corr_table %>%
  gt() %>%
  tab_header(
    title = "Pairwise Dynamic Conditional Correlations",
    subtitle = "DCC-GARCH estimated correlations"
  ) %>%
  cols_label(
    Pair = "Pair",
    Coef = "Coef.",
    Std_Error = "Std. Err.",
    z = "z",
    P_value = "P>|z|",
    CI_Lower = "95% CI Lower",
    CI_Upper = "95% CI Upper"
  ) %>%
  fmt_number(
    columns = c(Coef, Std_Error, z, P_value, CI_Lower, CI_Upper),
    decimals = 4
  ) %>%
  tab_options(
    table.font.size = 13,
    heading.title.font.size = 16,
    heading.subtitle.font.size = 13
  )

```

Table 2: Pairwise Dynamic Conditional Correlations

DCC-GARCH estimated correlations

Pair	Coef.	Std. Err.	z	P> z	95% CI Lower	95% CI Upper
corr(r_SH, r_Hang_Seng)	0.4973	0.0015	330.0839	0.0000	0.4944	0.5003
corr(r_SH, r_Nikkei)	0.2776	0.0012	230.8606	0.0000	0.2752	0.2799
corr(r_SH, r_KOSPI)	0.3063	0.0014	226.6344	0.0000	0.3036	0.3089
corr(r_SH, r_STI)	0.3070	0.0014	215.8423	0.0000	0.3042	0.3098
corr(r_SH, r_SP500)	0.1231	0.0011	112.4937	0.0000	0.1209	0.1252
corr(r_Hang_Seng, r_Nikkei)	0.5429	0.0012	470.5179	0.0000	0.5407	0.5452
corr(r_Hang_Seng, r_KOSPI)	0.6139	0.0010	628.8853	0.0000	0.6120	0.6158
corr(r_Hang_Seng, r_STI)	0.6623	0.0013	518.8291	0.0000	0.6597	0.6648
corr(r_Hang_Seng, r_SP500)	0.2610	0.0012	225.8682	0.0000	0.2587	0.2632
corr(r_Nikkei, r_KOSPI)	0.6232	0.0011	558.9445	0.0000	0.6210	0.6254
corr(r_Nikkei, r_STI)	0.5547	0.0010	555.2968	0.0000	0.5527	0.5566
corr(r_Nikkei, r_SP500)	0.2310	0.0010	224.5392	0.0000	0.2290	0.2331
corr(r_KOSPI, r_STI)	0.5691	0.0011	533.4580	0.0000	0.5670	0.5711
corr(r_KOSPI, r_SP500)	0.2502	0.0011	232.7425	0.0000	0.2481	0.2523
corr(r_STI, r_SP500)	0.2818	0.0012	235.3592	0.0000	0.2795	0.2842

. Create Daily Dynamic Correlation Dataset

```
# Extract correlations between Shanghai and other markets
corr_SH_Hang_Seng <- dcc_cor_with_sp500[1, 2, ]
corr_SH_Nikkei <- dcc_cor_with_sp500[1, 3, ]
corr_SH_KOSPI <- dcc_cor_with_sp500[1, 4, ]
corr_SH_STI <- dcc_cor_with_sp500[1, 5, ]
corr_SH_SP500 <- dcc_cor_with_sp500[1, 6, ]
```

```
# Create daily DCC correlation dataset
dcc_daily <- data.frame(
  Date = daily_returns$Date,
  corr_SH_Hang_Seng = corr_SH_Hang_Seng,
  corr_SH_Nikkei = corr_SH_Nikkei,
  corr_SH_KOSPI = corr_SH_KOSPI,
  corr_SH_STI = corr_SH_STI,
  corr_SH_SP500 = corr_SH_SP500
)
```

```
head(dcc_daily)
```

	Date	corr_SH_Hang_Seng	corr_SH_Nikkei	corr_SH_KOSPI
1970-01-02	2003-01-07	0.5056061	0.2876309	0.3159414
1970-01-03	2003-01-08	0.5056384	0.2878681	0.3135983
1970-01-04	2003-01-09	0.5002189	0.2568466	0.3059322
1970-01-05	2003-01-10	0.4955702	0.2530139	0.2671782
1970-01-06	2003-01-14	0.4927253	0.2544829	0.2684217
1970-01-07	2003-01-15	0.4820211	0.2630087	0.3204698

	corr_SH_STI	corr_SH_SP500
1970-01-02	0.3184949	0.1352652
1970-01-03	0.3183347	0.1357583
1970-01-04	0.3285003	0.1071317
1970-01-05	0.3279744	0.1223478
1970-01-06	0.3213080	0.1225158
1970-01-07	0.4053041	0.1256570

```
summary(dcc_daily)
```

Date	corr_SH_Hang_Seng	corr_SH_Nikkei	corr_SH_KOSPI
Min. :2003-01-07	Min. :0.1738	Min. :0.07161	Min. :0.05822
1st Qu.:2008-03-03	1st Qu.:0.4528	1st Qu.:0.22311	1st Qu.:0.23917
Median :2013-04-25	Median :0.5280	Median :0.27455	Median :0.31676
Mean :2013-05-02	Mean :0.4973	Mean :0.27758	Mean :0.30628
3rd Qu.:2018-07-05	3rd Qu.:0.5647	3rd Qu.:0.32847	3rd Qu.:0.36680
Max. :2023-08-31	Max. :0.6697	Max. :0.48863	Max. :0.55870

corr_SH_STI	corr_SH_SP500
Min. :0.03569	Min. :-0.06323
1st Qu.:0.24767	1st Qu.: 0.07442
Median :0.31201	Median : 0.12174
Mean :0.30703	Mean : 0.12306
3rd Qu.:0.37470	3rd Qu.: 0.16490
Max. :0.56620	Max. : 0.34965

11. Average Dynamic Conditional Correlations

Calculate average dynamic correlations between Shanghai and other markets

```
avg_shanghai_cor <- data.frame(
  Market_Pair = c(
    "Shanghai-Hang Seng",
    "Shanghai-Nikkei",
    "Shanghai-KOSPI",
    "Shanghai-STI",
    "Shanghai-SP500"
  ),
  Average_DCC_Correlation = c(
    mean(dcc_daily$corr_SH_Hang_Seng, na.rm = TRUE),
    mean(dcc_daily$corr_SH_Nikkei, na.rm = TRUE),
    mean(dcc_daily$corr_SH_KOSPI, na.rm = TRUE),
    mean(dcc_daily$corr_SH_STI, na.rm = TRUE),
    mean(dcc_daily$corr_SH_SP500, na.rm = TRUE)
  )
)
```



```

)
)

avg_shanghai_cor

      Market_Pair Average_DCC_Correlation
1 Shanghai-Hang Seng      0.4973318
2   Shanghai-Nikkei      0.2775815
3   Shanghai-KOSPI      0.3062762
4   Shanghai-STI      0.3070326
5   Shanghai-SP500      0.1230622

## 11. Average Dynamic Conditional Correlations

# Calculate the average dynamic conditional correlation matrix
avg_dcc_cor_matrix_with_sp500 <- apply(dcc_cor_with_sp500, c(1, 2), mea
n, na.rm = TRUE)

# Add market names
colnames(avg_dcc_cor_matrix_with_sp500) <- colnames(dcc_data_with_sp500)
rownames(avg_dcc_cor_matrix_with_sp500) <- colnames(dcc_data_with_sp500)

# Display the full average DCC correlation matrix
knitr::kable(
  avg_dcc_cor_matrix_with_sp500,
  digits = 4,
  caption = "Average Dynamic Conditional Correlation Matrix"
)

```

Average Dynamic Conditional Correlation Matrix

	r_Shanghai	r_Hang_Seng	r_Nikkei_225	r_KOSPI	r_STI	r_SP500
r_Shanghai	1.0000	0.4973	0.2776	0.3063	0.3070	0.1231
r_Hang_Seng	0.4973	1.0000	0.5429	0.6139	0.6623	0.2610
r_Nikkei_225	0.2776	0.5429	1.0000	0.6232	0.5547	0.2310
r_KOSPI	0.3063	0.6139	0.6232	1.0000	0.5691	0.2502
r_STI	0.3070	0.6623	0.5547	0.5691	1.0000	0.2818
r_SP500	0.1231	0.2610	0.2310	0.2502	0.2818	1.0000

11. Export Daily Dynamic Correlations

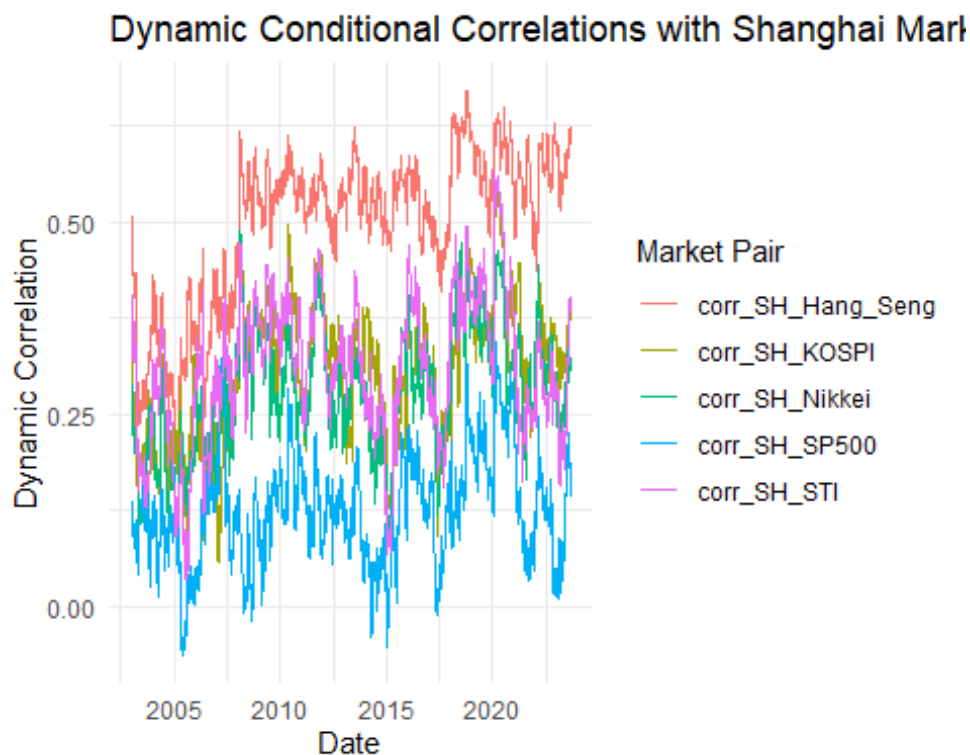
```
write.csv(dcc_daily, "dcc_dynamic_correlations_daily.csv", row.names = FALSE)
```

12. Plot Daily Dynamic Correlations

```
library(ggplot2)
library(tidyr)

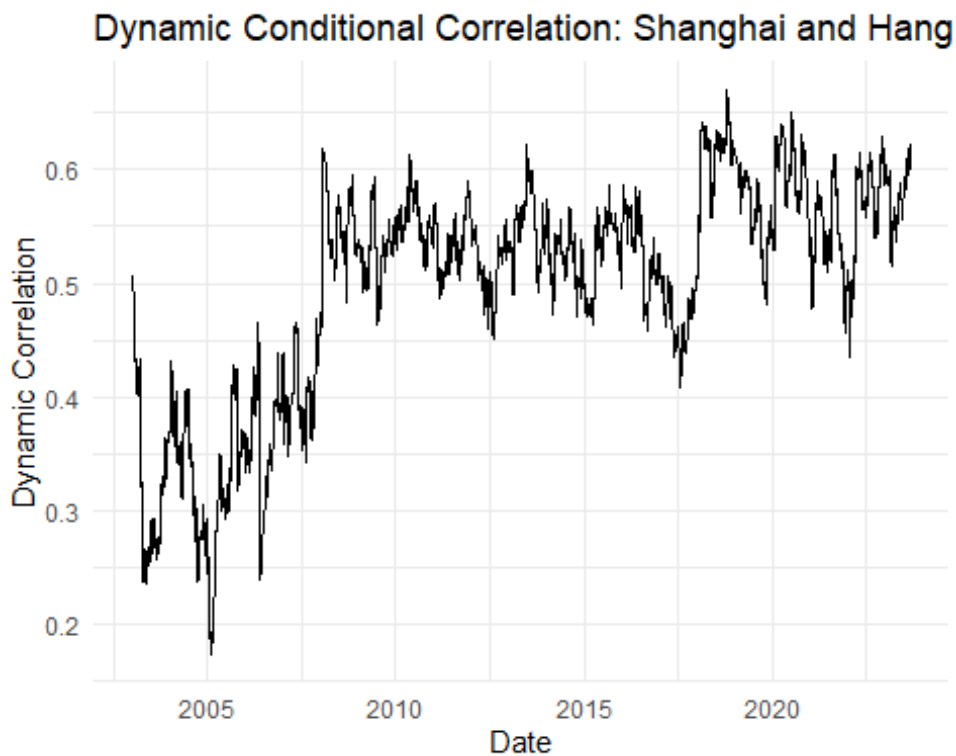
dcc_long <- dcc_daily %>%
  pivot_longer(
    cols = starts_with("corr_"),
    names_to = "Market_Pair",
    values_to = "Correlation"
  )

ggplot(dcc_long, aes(x = Date, y = Correlation, color = Market_Pair)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlations with Shanghai Market",
    x = "Date",
    y = "Dynamic Correlation",
    color = "Market Pair"
  ) +
  theme_minimal()
```

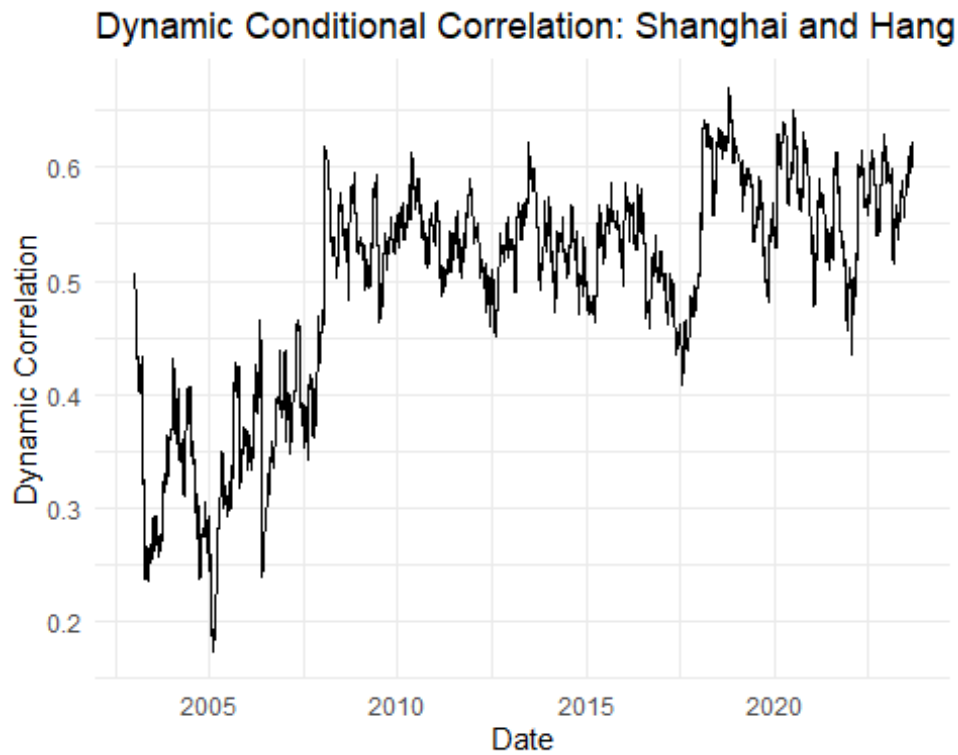


```
## 12.1 Plot Shanghai-Hang Seng Dynamic Correlation
```

```
ggplot(dcc_daily, aes(x = Date, y = corr_SH_Hang_Seng)) +  
  geom_line(linewidth = 0.5) +  
  labs(  
    title = "Dynamic Conditional Correlation: Shanghai and Hang Seng",  
    x = "Date",  
    y = "Dynamic Correlation"  
  ) +  
  theme_minimal()
```

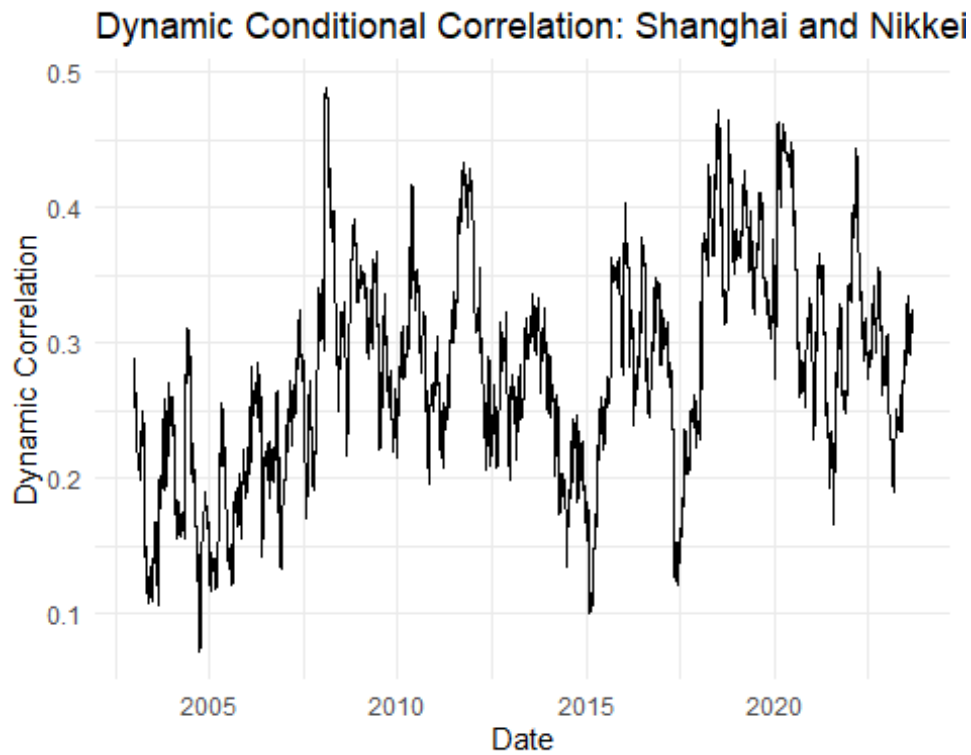


```
# Shanghai-Hang Seng  
plot_sh_hs <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_Hang_Seng)) +  
  geom_line(linewidth = 0.5) +  
  labs(  
    title = "Dynamic Conditional Correlation: Shanghai and Hang Seng",  
    x = "Date",  
    y = "Dynamic Correlation"  
  ) +  
  theme_minimal()  
  
plot_sh_hs
```



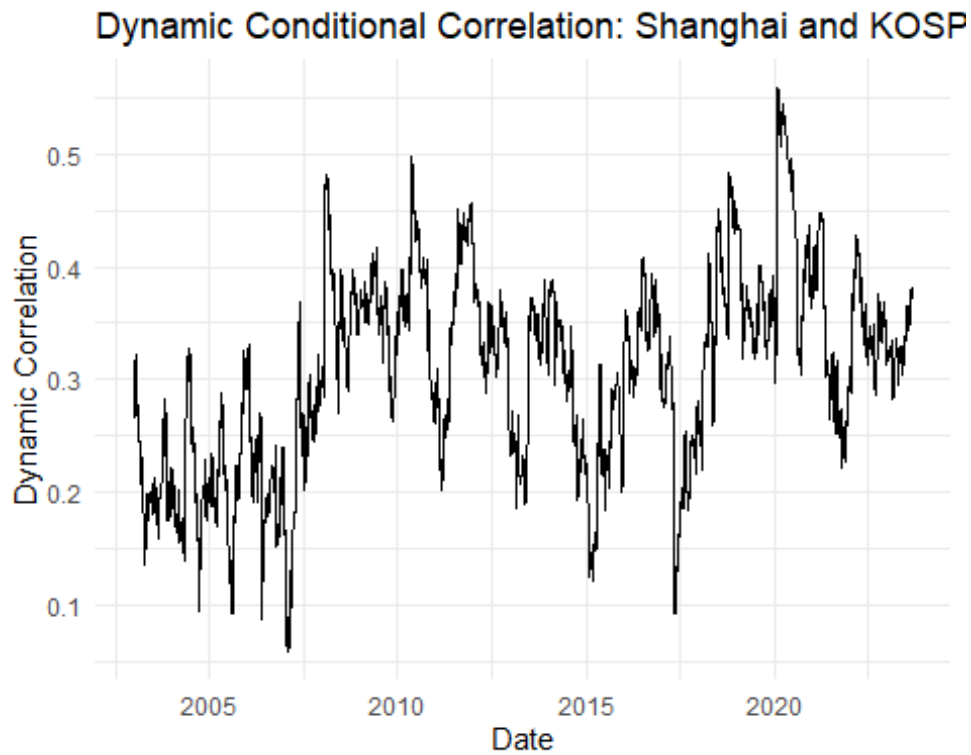
```
# Shanghai-Nikkei
plot_sh_nikkei <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_Nikkei))
+
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and Nikkei 225",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_nikkei
```



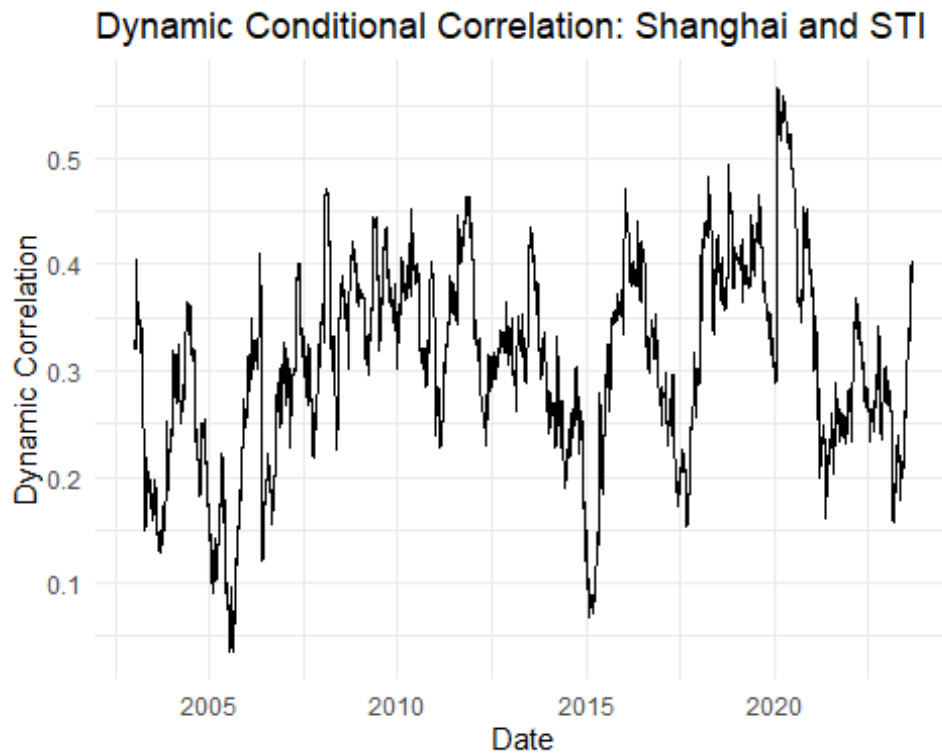
```
# Shanghai-KOSPI
plot_sh_kospi <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_KOSPI)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and KOSPI",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_kospi
```



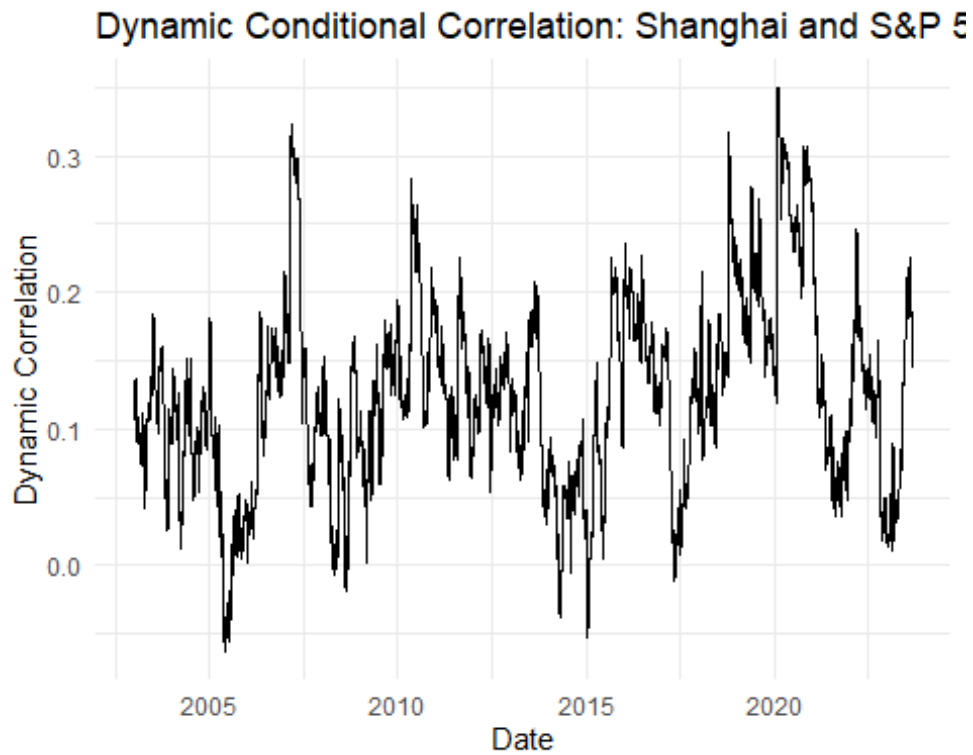
```
# Shanghai-STI
plot_sh_sti <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_STI)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and STI",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_sti
```



```
# Shanghai-S&P 500
plot_sh_sp500 <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_SP500)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and S&P 500",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_sp500
```



```
ggplot(dcc_daily, aes(x = Date, y = corr_SH_Nikkei)) +  
  geom_line(linewidth = 0.5) +  
  labs(  
    title = "Dynamic Conditional Correlation: Shanghai and Nikkei 225",  
    x = "Date",  
    y = "Dynamic Correlation"  
  ) +  
  theme_minimal()
```


Dynamic Conditional Correlation: Shanghai and Nikkei

